COLLEGE ACADEMIC PROJECT REPORT

SOUNDSPHERE – MUSIC STREAMING WEB APPLICATION

Project Metadata

- **Project Title**: SoundSphere – Music Streaming Web Application
- **Course**: Front-End Web Development / Minor Project
- **Technology Stack**: React.js, Vite, JavaScript (ES6+), CSS3, React Router DOM, Lucide React
- **GitHub Repository**: https://github.com/aniruthramidi/soundsphere
- **Academic Year**: 2026–2027
- **Deliverable Version**: 1.0.0

EXECUTIVE ABSTRACT

SoundSphere is an immersive, modern, commercial-grade music streaming web application developed as a college front-end development project. The application delivers a high-performance audio discovery and streaming platform inspired by modern web interfaces.

Built using **React.js 18**, **Vite**, **JavaScript (ES6+)**, **CSS3**, **React Router DOM**, and **Lucide React**, SoundSphere provides client-side routing without page reloads, an asynchronous service layer simulating REST APIs with Promises, and a persistent HTML5 audio player that maintains uninterrupted playback during page navigation.

Designed with a sleek **Monochrome (Black & White)** theme, SoundSphere prioritizes high visual contrast, WCAG accessibility compliance, dynamic search filtering, user listening analytics, and local storage state persistence. This report details the architecture, design choices, component structure, testing results, and assignment requirement coverage of the project.

CHAPTER 1: INTRODUCTION & OBJECTIVES

1.1 Background & Motivation

Modern web applications demand high interactivity, fast initial load times, and fluid page transitions. Music streaming applications represent a complex front-end challenge, requiring persistent audio playback state while users navigate between different views, filter content, and search for tracks in real-time.

1.2 Project Objectives

The main objectives of the SoundSphere project are to:

1. Develop a responsive single-page application (SPA) using **React** and **Vite**.
2. Implement **React Router** for seamless multi-page navigation without page reloads.
3. Design a persistent **HTML5 Audio Player** component operating under a global React Context provider.
4. Abstract backend data operations into an asynchronous **Service Layer** (`musicService.js`) using Promises.
5. Create four interconnected views: **Home Page**, **User Dashboard**, **Discover/Search Page**, and **Music Detail Page**.
6. Enforce accessibility (a11y) standards, keyboard navigation, and responsive CSS grid/flexbox layouts.

7. Provide comprehensive documentation including visual wireframes, usability test matrix, and GitHub repository integration.

CHAPTER 2: TECHNOLOGY STACK & ARCHITECTURE

2.1 Technology Stack Selection

Component	Selected Technology	Rationale / Purpose
Core Library	**React.js 18**	Functional component architecture, declarative UI, custom hooks (`useState`, `useEffect`, `useContext`, `useRef`).
Build Tool	**Vite v8.3**	Lightning-fast HMR (Hot Module Replacement), optimized production bundling, minimal config overhead.
Language	**JavaScript (ES6+)**	Modern JavaScript standard using async/await, Promises, array methods, destructuring, and ES modules.
Routing	**React Router DOM v7**	Client-side routing (`BrowserRouter`, `Routes`, `Route`, `Link`, `NavLink`, `useParams`, `useSearchParams`).
Icons	**Lucide React**	Scalable, accessible SVG icons (`Play`, `Pause`, `Heart`, `Share2`, `Search`, `Compass`, `Headphones`).
Styling	**Vanilla CSS3**	Custom CSS variables (`:root`), monochrome color tokens, CSS Grid, Flexbox, media queries (`@media`).
State Management	**React Context API**	Global player state (`PlayerContext.jsx`) managing queue, active track, volume, mute, and playlists.
Persistence	**LocalStorage API**	Browser persistent storage for liked track IDs and user created playlists across sessions.

2.2 System Directory Structure

```
SoundSphere/
│
├── public/                    # Static assets & generated album artwork
│   ├── artwork/               # High-definition 1:1 real album covers
│   ├── favicon.svg            # SVG brand icon
│   └── icons.svg              # Vector icons
│
├── src/
│   ├── assets/                # Brand assets
│   ├── components/            # Reusable UI React Components
│   │   ├── Navbar.jsx         # Top brand header & responsive drawer
│   │   ├── MusicCard.jsx      # Interactive track card with play/like/share
│   │   ├── TrackList.jsx      # Row-based table view of tracks
│   │   ├── MusicPlayer.jsx    # Persistent fixed audio player (<audio>)
│   │   ├── SearchBar.jsx      # Instant search input with clear trigger
│   │   ├── GenreChip.jsx      # Interactive genre filter chip
```

```
│   │   ├── PlaylistCard.jsx    # Playlist showcase card
│   │   ├── Loading.jsx         # Skeleton & spinner loading indicator
│   │   ├── ErrorMessage.jsx    # Graceful error state card
│   │   ├── Footer.jsx          # Commercial brand footer & social links
│   │   ├── Toast.jsx           # Temporary popup notification banner
│   │   └── PlaylistModal.jsx   # Add to playlist selection modal
│   │
│   ├── context/
│   │   └── PlayerContext.jsx   # React Context API global state provider
│   │
│   ├── pages/                  # Main Page Views
│   │   ├── HomePage.jsx        # Landing page with hero & featured music
│   │   ├── Dashboard.jsx       # User stats, favorites & custom playlists
│   │   ├── Discover.jsx        # Dynamic search & genre filter view
│   │   └── TrackDetail.jsx     # Param route (/track/:id) detail page
│   │
│   ├── services/
│   │   └── musicService.js     # Asynchronous API service layer (Promises)
│   │
│   ├── data/
│   │   └── tracks.js           # 15 sample tracks across 7 music genres
│   │
│   ├── App.jsx                 # Main layout & route definitions
│   ├── main.jsx                # Entry point
│   └── styles.css              # Global monochrome design system
│
├── docs/
│   ├── wireframes/             # SVG Wireframe diagrams & WIREFRAMES.md
│   ├── testing/                # USABILITY_TESTING.md (15 test cases)
│   └── PROJECT_REPORT.md       # Full Academic Project Report
│
├── index.html                  # HTML5 accessible root document
├── package.json                # Project dependencies & npm scripts
├── README.md                   # Complete GitHub project documentation
└── .gitignore                  # Git tracking exclusion rules
```

CHAPTER 3: SYSTEM IMPLEMENTATION & COMPONENT DESIGN

3.1 Service Layer Architecture (`musicService.js`)

To avoid embedding data logic directly within UI components, SoundSphere uses a dedicated service layer that simulates backend REST API endpoints using asynchronous Promises with realistic network latency (`setTimeout` 200–300ms).

Key Service Functions:

- `getTracks()` : Returns all available songs from storage or defaults.
- `getTrackById(id)` : Returns detailed track object matching numeric ID.
- `searchTracks(query)` : Filters tracks matching title, artist, album, or genre.
- `getTracksByGenre(genre)` : Returns tracks belonging to a specific genre category.
- `toggleLike(id)` : Toggles liked state of a track with local storage persistence.

- `getUserPlaylists()` & `addTrackToPlaylist(playlistId, trackId)`: Manages user custom playlists.

3.2 Global Player State Management (`PlayerContext.jsx`)

SoundSphere utilizes the React Context API to wrap the entire application in a single `PlayerProvider`. This context retains the HTML5 `<audio>` element reference and provides global methods:

- `currentTrack` & `isPlaying`: Tracks active song and playback status.
- `playTrack(track, queue)`: Loads song URL into audio player and starts playback.
- `togglePlayPause()`, `nextTrack()`, `prevTrack()`: Controls queue navigation.
- `seek(time)`: Updates audio `currentTime` dynamically.
- `volume` & `isMuted`: Controls audio gain and mute state.
- `likedTrackIds` & `userPlaylists`: Synchronizes user preferences across views.

3.3 Page Views & Interactivity

1. Page 1 – Landing / Home Page (`/`)

- **Hero Section**: High-impact banner with bold typography, "Start Listening" CTA, "Explore Discover" secondary action, and glowing visual badge.
- **Featured Music Grid**: Displays top played track cards with hover play overlays.
- **Trending Tracks**: Table view highlighting current top trending tracks.
- **Genre Banner Cards**: Monochrome gradient cards for rapid category filtering.
- **Popular Artists Showcase**: Avatar cards highlighting featured creators.

2. Page 2 – User Dashboard (`/dashboard`)

- **Profile Banner**: Personalized header ("Welcome back, Alex! - PRO MEMBER").
- **Listening Statistics (4 Cards)**:
  - *Songs Played*: 1,420
  - *Hours Listened*: 84.5 hrs
  - *Favorite Genre*: Electronic
  - *Playlists*: 8
- **Recently Played**: Carousel of recently listened tracks.
- **Favorite Songs**: Liked songs list with one-click "Play All" trigger.
- **Playlists**: Custom user created playlists display.

3. Page 3 – Music Detail Page (`/track/:id`)

- **Param Route Navigation**: Dynamically fetches track metadata based on URL parameter `:id`.
- **Large Artwork & Metadata**: High-resolution album cover, track title, artist, album, genre tag, release year, duration, and total play count.
- **Action Toolbar**: Large Play/Pause button, Like heart toggle, Web Share API / Copy Link button, and Add to Playlist modal trigger.
- **Track Narrative**: Descriptive story behind the composition.
- **Related Songs**: Tracks matching the current song's genre.

4. Page 4 – Discover / Search Page (`/discover`)

- **Dynamic Search Bar**: Instant real-time filtering across titles, artists, albums, and genres.
- **Genre Filter Chips**: Horizontal pill filters ("All", "Pop", "Rock", "Hip Hop", "Electronic", "Classical", "Indie", "Jazz").
- **View Switcher**: Toggle between Grid view and List table view modes.
- **URL Synchronization**: Syncs search states with browser search parameters (`?genre=Pop`, `?query=...`).

CHAPTER 4: UI DESIGN, ACCESSIBILITY & RESPONSIVE DESIGN

4.1 Monochrome Design System

The application adheres to a clean **Monochrome (Black & White)** design system defined via CSS variables in `:root`:

```css
:root {
  --background: #050505;        /* Deep obsidian black */
  --surface: #121212;           /* Dark charcoal surface */
  --surface-hover: #1a1a1a;     /* Elevated hover state */
  --surface-card: #161616;      /* Solid card background */
  --surface-border: rgba(255, 255, 255, 0.14);

  --primary: #ffffff;           /* Stark white text & primary actions */
  --secondary: #a1a1aa;         /* Muted silver gray */
  --accent: #d4d4d8;            /* Bright platinum */
  --player-bg: #0a0a0a;         /* Fixed player bar background */
}
```

4.2 Responsive Layout Adaptations

Using CSS Grid, Flexbox, and Media Queries (`@media`), SoundSphere adapts seamlessly across viewports:

- **Desktop (>=1024px)**: Multi-column grid, detailed track list tables, side-by-side hero sections.
- **Tablet (768px - 1023px)**: 2-column card layouts, stacked detail hero section.
- **Mobile (<768px)**: Collapsible hamburger menu drawer, single-column stacked cards, compact touch-friendly audio player controls.

4.3 Accessibility Compliance (a11y)

- **Semantic HTML5**: Uses `<header>` , `<nav>` , `<main>` , `<section>` , `<article>` , `<button>` , and `<footer>` .
- **ARIA Attributes**: `aria-label` , `aria-expanded` , `aria-pressed` , `aria-modal` , and `role="region"` .
- **Keyboard Navigation**: Full `Tab` focus ring support with `:focus-visible` styling and `Enter` / `Space` activation.
- **Image Alt Text**: Descriptive `alt` attributes on all album covers and artist avatars.

CHAPTER 5: TESTING & USABILITY RESULTS

A formal usability test suite consisting of 15 test cases was conducted to evaluate navigation integrity, playback functions, state persistence, and responsive UI behavior.

5.1 Usability Test Matrix

Test ID	Category	Action / Steps	Expected Result	Status
TC-01	Navigation	Click "Dashboard" in Navbar	Route opens `/dashboard` without page reload.	**PASS**

TC-02	Navigation	Click "Discover" in Navbar	Route opens `/discover` showing search bar & chips.	**PASS**
TC-03	Search	Type "Aetheria" into Search Bar	Real-time filter displays matching tracks only.	**PASS**
TC-04	Playback	Click "Play" on song card	Audio player loads song artwork, title, and plays audio.	**PASS**
TC-05	Playback	Click "Pause" on audio player	Playback pauses instantly; icon switches to Play.	**PASS**
TC-06	Controls	Click "Next Track" button	Player skips to next queued song automatically.	**PASS**
TC-07	Controls	Click "Previous Track" button	Player skips backward to previous queued song.	**PASS**
TC-08	Favorites	Click Heart icon on any track	Song liked state toggles; updates localStorage & toast.	**PASS**
TC-09	Share	Click Share icon on track	Web Share API opens or link is copied to clipboard.	**PASS**
TC-10	Genre Filter	Click "Pop" genre chip	View filters to Pop songs; updates URL `?genre=Pop`.	**PASS**
TC-11	Detail View	Click track title/artwork	Route opens `/track/:id` displaying full details & story.	**PASS**
TC-12	Playlists	Click "Add to Playlist" button	Modal opens to choose or create new user playlist.	**PASS**
TC-13	Audio Seeking	Drag progress slider	Audio playback position jumps to selected timestamp.	**PASS**
TC-14	Responsive	Resize screen below 768px	Navbar transforms to mobile drawer; layout stacks.	**PASS**
TC-15	Keyboard a11y	Navigate via `Tab` and `Enter`	Focus indicators stay visible; buttons execute properly.	**PASS**

5.2 Testing Summary

- **Total Test Cases**: 15
- **Passed**: 15
- **Failed**: 0
- **Pass Rate**: 100%

CHAPTER 6: ASSIGNMENT REQUIREMENT COVERAGE

Requirement	Specification	Implementation in SoundSphere	Coverage Status

1. Front-end Application	React, Vite, HTML5, CSS3, JS ES6+	Built using React 18 + Vite	**100% Satisfied**
2. Tech Stack Constraints	React Router, Lucide React (No TS)	Implemented without TypeScript	**100% Satisfied**
3. 4 Required Views	Home, Dashboard, Detail, Discover	All 4 views implemented & linked	**100% Satisfied**
4. Navigation	React Router without page reloads	Client-side routing configured	**100% Satisfied**
5. Service Layer	`musicService.js` returning Promises	Async service layer with delay	**100% Satisfied**
6. Reusable Components	Navbar, Cards, Player, Search, etc.	12 modular components created	**100% Satisfied**
7. Persistent Player	HTML5 `<audio>` player at bottom	Fixed player synchronized in Context	**100% Satisfied**
8. Interactive Features	Like, Share, Add Playlist, Search	Fully functional interactive controls	**100% Satisfied**
9. UI Design System	Modern dark/monochrome layout	Custom CSS variables & cards	**100% Satisfied**
10. Responsive Layout	Mobile, tablet & desktop support	CSS media queries implemented	**100% Satisfied**
11. Accessibility	ARIA tags, focus states, semantic HTML	WCAG compliant implementation	**100% Satisfied**
12. Wireframes	`docs/wireframes/` with SVG files	SVG diagrams & `WIREFRAMES.md`	**100% Satisfied**
13. Usability Testing	`docs/testing/USABILITY_TESTING.md`	15 detailed test cases	**100% Satisfied**
14. GitHub Readiness	Git commands & clean repo structure	Pushed to GitHub repository	**100% Satisfied**

CHAPTER 7: CONCLUSION & FUTURE ENHANCEMENTS

7.1 Conclusion

The **SoundSphere** music streaming application successfully satisfies all technical, functional, architectural, and design requirements set forth for the college assignment. The project demonstrates strong skills in modern component-based React development, state management, asynchronous JavaScript service design, responsive CSS layouts, accessibility standards, and software documentation.

7.2 Future Enhancements

1. **Real REST / GraphQL Backend**: Integration with Node.js/Express or Firebase backend.
2. **User Authentication**: Secure JWT or OAuth user sign-in & profile management.
3. **Cloud Audio Storage**: Integration with Google Cloud Storage / AWS S3 for user uploads.
4. **AI Music Recommendations**: Machine learning recommendation engine based on listening history.

CHAPTER 8: REFERENCES & APPENDIX

1. **GitHub Repository**: https://github.com/aniruthramidi/soundsphere
2. **React Documentation**: https://react.dev
3. **Vite Guide**: https://vite.dev
4. **React Router v7**: https://reactrouter.com
5. **Lucide Icons**: https://lucide.dev
6. **MDN Web Docs (HTML5 Audio API)**: https://developer.mozilla.org

---

Report Compiled & Generated for SoundSphere Front-End Development Assignment submission.